

PROJECT
REPORT



- C# Desktop Application
- Windows Forms UI
- JSON Data Storage
- PDF Resume Export
- Multiple Templates

Resume Builder
Documentation Package

COMPLETE PROJECT DOCUMENTATION

Resume Builder Application

A concise technical report presenting the structure, design decisions, core components, and future scope of a desktop-based resume creation system.

Prepared For	Ma'am Amna
Prepared By	1. Muhammad Hammad (2023-uam-2046) 2. Muneeb Hasan Khan (2023-uam-2044)
Date	_____

06 Templates	PDF Generation	JSON Storage
-----------------	-------------------	-----------------

Table of Contents

Project Overview	3
Project Objectives	3
Technologies Used	3
Project Architecture	4
Folder Structure Analysis	4
Entry Point Analysis	5
Business Logic Layer (BLL)	5
Data Access Layer (DAL)	6
Models Analysis	6
Helper Classes Analysis	6
PDF Generation System	6
Template System Analysis	7
Detailed Template Analysis	8
User Interface (UI) Analysis	9
Event-Driven Programming Concepts	9
Object-Oriented Programming Concepts Used	10
Data Flow Analysis	10
Features of the Application	11
Advantages of the System	11
Limitations of the Current System	11
Future Improvements	11
Security and Validation	12
Software Engineering Principles Applied	12
Key Libraries and Namespaces	12
Overall Project Evaluation	13
Conclusion	13

Project Overview

This section presents a complete overview of the Resume Builder Application, including its purpose, scope, architecture, major modules, technologies, and overall functionality. The project is designed as a desktop-based solution that helps users create, save, preview, and export professional resumes using structured forms, reusable templates, JSON-based storage, and PDF generation.

- **User Interface (UI):** Manages user interaction and form-based workflows.
- **Business Logic Layer (BLL):** Processes resume content and coordinates document generation.
- **Data Access Layer (DAL):** Handles JSON-based persistence and retrieval.
- **Models:** Define the core data entities used across the application.
- **PDF Generation Engine:** Produces export-ready resume documents.
- **Resume Templates:** Provide multiple presentation styles for final output.
- **Utility Helpers:** Support validation, file handling, and image processing.

The project uses the iText PDF Library to generate modern, template-driven resume layouts that are consistent, scalable, and suitable for professional use.

Project Objectives

The project objectives describe the main goals achieved by the application and show how the system provides a complete resume-building workflow from data entry to final PDF output.

Objective Area	Description
Resume Creation	Enable users to create professional resumes efficiently.
Templates	Provide multiple resume templates for varied presentation styles.
Export	Generate resumes in PDF format for professional distribution.
Storage	Save and reload resume data using JSON for reuse and portability.
Quality	Validate input, support image handling, and maintain scalable architecture.

Technologies Used

This section identifies the primary technologies, frameworks, and libraries used to implement the application and explains the role of each within the overall solution.

Technology	Purpose
C#	Primary programming language.
Windows Forms	Desktop user interface development.

.NET	Application framework and runtime support.
iText PDF Library	PDF generation and layout processing.
JSON Serialization	Structured data storage and retrieval.
Visual Studio	Integrated development environment.
System.Drawing	Image processing and profile picture support.

Project Architecture

The application follows a layered architecture to separate presentation logic, business rules, data persistence, and document generation. This structure improves maintainability, supports reuse, and simplifies future extension.

Architectural Layers

Layer	Role
Presentation Layer	Captures input and manages navigation across forms.
Business Logic Layer	Processes input, creates resume objects, and coordinates operations.
Data Access Layer	Saves and reloads structured resume data using JSON files.
Models / Storage	Represents domain entities used for persistence and export.

Folder Structure Analysis

Main Directories

The project is organized into clearly separated directories, each responsible for a distinct part of the application. This structure supports modular development and simplifies maintenance.

Folder	Purpose
BLL	Implements business rules and processing logic.
DAL	Manages data storage and retrieval.
Helpers	Provides reusable utility methods.
Models	Defines core domain entities and data structures.
PDF	Contains the PDF generation engine.
Templates	Stores template implementations and formatting logic.
UI	Contains Windows Forms interfaces and user workflows.

Entry Point Analysis

Program.cs

Purpose

This file serves as the main entry point of the application and is responsible for launching the Windows Forms environment.

Key Responsibilities

- Initializes the Windows Forms environment.
- Starts the application.
- Opens the MainForm.

Startup Workflow

- The application process starts.
- The Windows Forms environment is initialized.
- The MainForm is launched as the primary interface.

Important Code Concept

```
Application.Run(new MainForm());
```

This statement launches the main application window and begins the user-facing workflow.

Business Logic Layer (BLL)

The Business Logic Layer contains the core processing logic of the application. It transforms validated user input into structured resume data, manages templates, and coordinates document-generation workflows.

Class	Purpose	Main Functions	Responsibility
ResumeManager.cs	Creates and prepares Resume objects from user input.	CreateResume()	Builds structured resume data for storage, preview, or PDF generation.
PDFManager.cs	Coordinates PDF generation and preview workflows.	GeneratePDF(), GeneratePDFBytes()	Connects resume data, templates, PDFGenerator, and preview support.
TemplateManager.cs	Registers and provides available resume templates.	GetAvailableTemplates()	Centralizes template access using a factory-like approach.

Responsibilities

Data Access Layer (DAL)

The Data Access Layer is responsible for storing and retrieving resume data in a lightweight, portable format. It uses JSON files to maintain structured records without requiring a database system.

Class	Purpose	Features	Benefit
JsonStorageHelper.cs	Saves and loads resume data using JSON serialization.	Save, load, serialize, deserialize; WriteIndented and case-insensitive options.	Provides lightweight, portable, human-readable storage.

Models Analysis

The model layer defines the core data structures that represent resume information throughout the application. These classes ensure consistency across creation, storage, preview, and PDF export workflows.

Helper Classes Analysis

Helper classes provide reusable utility functions that support common application tasks such as file handling, input validation, and image processing. Their use improves maintainability, reduces code duplication, and keeps business logic focused on core operations.

Helper	Purpose	Features / Functions	Benefit
FileHelper.cs	Manages file dialogs and file path selection.	Open/save dialogs, GetSavePath(), GetOpenPath(), path management.	Simplifies reusable file handling.
ValidationHelper.cs	Validates resume data before storage or export.	Empty field checks, email validation, resume validation, regex-based format checking.	Improves data quality and user experience.
ImageHelper.cs	Processes profile images for resume templates.	Resize, convert, improve quality; uses Bitmap, Graphics, and high-quality interpolation.	Ensures images fit cleanly in PDF templates.

PDF Generation System

The PDF generation subsystem is responsible for transforming structured resume data into polished, export-ready documents. It combines the selected template, formatting rules, and scaling logic to produce consistent PDF output.

Component	Purpose	Responsibilities	Benefit
PDFGenerator.cs	Generates resume PDFs from structured data and selected templates.	Create PDF, apply template, generate output, handle scaling, use iText.	Produces professional, scalable, template-based PDF output.
Dynamic Scaling	Adjusts content size to fit available layout space.	Uses FindOptimalScale() to reduce overflow and preserve readability.	Improves consistency across templates and resume lengths.
Workflow	Converts resume data into final PDF output.	Prepare data, select template, apply layout, preview or save final PDF.	Supports fast generation and multiple template options.

Template System Analysis

The template system enables the application to offer multiple resume layouts through a consistent generation workflow. It defines how templates are described, scaled, and formatted during PDF export.

IResumeTemplate.cs

Purpose

IResumeTemplate defines the common contract that each resume template must follow to work within the PDF-generation pipeline.

Interface Members

Member	Purpose
TemplateName	Stores the template's display name.
Description	Briefly explains the template's style or use case.
ApplyTemplate()	Applies the template's layout and styling to the PDF.

Benefits of the Interface

- Standardization
- Extensibility
- Loose coupling
- Easy maintenance

IScalableTemplate Interface

Purpose

IScalableTemplate extends the template model by enabling adjustable content scaling.

Property

```
float Scale { get; set; }
```

Benefit

This interface lets templates dynamically adjust content density, improving layout flexibility for resumes of varying lengths.

ParagraphExtensions.cs

Purpose

ParagraphExtensions.cs provides reusable extension methods that simplify paragraph formatting during PDF generation.

Main Extension

- **SetBold():** Applies bold formatting to paragraphs.

Concept Used

Extension Methods in C#.

Benefit

This reduces repeated styling code and improves maintainability by centralizing common paragraph-formatting behavior.

Detailed Template Analysis

This chapter reviews the six resume templates included in the application. Each template offers a distinct visual style, layout approach, and professional use case, helping users choose a format that matches their goals.

Template	Style	Features	Suitable For
Template1 – Classic	Traditional single-column layout	Serif styling, professional formatting, clean sections, elegant lines	Corporate resumes, formal jobs, academic profiles
Template2 – Modern	Modern two-column layout	Sidebar design, progress bars, dark slate theme, visual skill indicators	Modern companies, software developers, designers
Template3 – Creative	Portfolio-style layout	Teal sidebar, peach header, circular profile image, portfolio appearance	Creative professionals, graphic designers, artists
Template4 – Minimal	Simple and clean layout	Light colors, minimal sections, balanced spacing, modern structure	Students, fresh graduates, simple professional resumes
Template5 – Professional	Business-oriented layout	Slate-blue sidebar, teal accents, professional	Business professionals, managers, executives

		formatting, strong readability	
Template6 – Elegant	Elegant single-column layout	Navy blue accents, clean typography, balanced spacing, sophisticated look	Executive resumes, senior professionals, consultants

User Interface (UI) Analysis

The application uses Windows Forms to provide a structured, event-driven graphical interface for resume creation, template selection, preview, and export workflows.

Form	Purpose	Features	Responsibility
MainForm.cs	Main navigation hub.	Create new resume, load resume, exit application.	Uses button click events for navigation.
ResumeForm.cs	Main resume data-entry interface.	Personal details, education, experience, skills, image upload, save, validation.	Collects input, populates models, and calls business logic.
TemplateSelectionForm.cs	Allows users to choose a resume template.	Preview templates, navigate options, generate PDF, open preview.	Connects template choice with PDF generation.
PreviewForm.cs	Displays generated resume previews.	Preview PDF, save final PDF, open generated file.	Handles temporary preview and export support.

Event-Driven Programming Concepts

The application relies heavily on event-driven programming, where user actions such as button clicks trigger specific workflows and interface updates.

Examples

Button Click Events

```
btnCreateNew.Click += BtnCreateNew_Click;
```

Importance

- User interaction handling
- Responsive UI
- Better user experience

Object-Oriented Programming Concepts Used

The project demonstrates several core object-oriented programming concepts that contribute to its modularity, readability, and extensibility.

Encapsulation

Classes keep related data and methods together.

- **Example:**
 - Resume class
 - Education class
 - Experience class

Abstraction

Interfaces hide implementation details.

- **Example:**

```
IResumeTemplate
```

Inheritance

Templates implement shared interfaces.

- **Example:**

```
Template1 : IScalableTemplate
```

Polymorphism

Different templates behave differently using the same interface.

- **Example:**

```
ApplyTemplate()
```

Data Flow Analysis

Complete Application Workflow

Step	Process
Start	Application opens MainForm.
Input	User creates or loads a resume and enters details in ResumeForm.
Validation	ValidationHelper checks input quality and completeness.
Processing	ResumeManager builds a structured Resume object.
Output	TemplateSelectionForm, PDFManager, PDFGenerator, and PreviewForm complete PDF generation and saving.

Features of the Application

Core Features

- **Resume Creation:** Users can create resumes with structured data.
- **Multiple Templates:** Six professionally designed templates are available.
- **PDF Export:** Resumes can be exported as PDF files.
- **JSON Save/Load:** Users can save and reload resume data.
- **Image Support:** Profile images are supported.
- **Validation:** Input validation prevents incorrect data.
- **Preview System:** Users can preview resumes before saving.

Advantages of the System

Category	Advantages
User Experience	User-friendly interface, modern UI structure, and preview support.
Output Quality	Professional PDF output with multiple template options.
Architecture	Reusable architecture, good separation of concerns, and easy maintenance.
Data Handling	Portable JSON storage and scalable template system.
Growth	Extensible design suitable for future improvements.

Limitations of the Current System

Limitation Area	Current Limitation
Platform	Desktop-only application with no web or cloud version.
Storage	No cloud storage, online resume sharing, or database integration.
Customization	Limited customization options, font control, and no drag-and-drop editor.
Automation	No AI-generated content suggestions.

Future Improvements

Improvement Area	Suggested Enhancements
AI Integration	AI summaries, skill recommendations, and resume optimization.
Cloud Features	Online storage, resume synchronization, and account system.

Export Formats	DOCX, HTML, and image export support.
Customization	Custom colors, custom fonts, and drag-and-drop sections.
ATS Optimization	ATS score analysis, keyword optimization, and resume scanning.
Web Version	Convert into an ASP.NET web app or React + API application.

Security and Validation

Area	Controls / Practices
Validation Features	Empty field checks, email format validation, and data consistency checks.
File Handling Security	Controlled file dialogs, structured storage, and safe serialization.

Software Engineering Principles Applied

Principle	Implementation
Separation of Concerns	Layered architecture
Reusability	Helper classes
Scalability	Template interface system
Maintainability	Modular structure
Abstraction	Interfaces
Encapsulation	Model classes
Extensibility	Template manager

Key Libraries and Namespaces

Important Namespaces

Namespace	Purpose
System.Windows.Forms	UI creation
System.Text.Json	JSON serialization
iText.Kernel.Pdf	PDF creation
iText.Layout	PDF layout
System.Drawing	Image processing

Overall Project Evaluation

The Resume Builder Application is a well-structured desktop project that demonstrates sound software engineering practice, modular design, and practical document-generation capabilities.

Evaluation Area	Strength
Programming	Strong object-oriented programming concepts and modular design.
Architecture	Layered software architecture with reusable components.
Output	Professional PDF generation through a template-based rendering system.
Usability	User-friendly interface suitable for academic and portfolio use.

Suitable Contexts	Development Areas Covered
Academic submissions	UI development
Final year projects	Data management
Portfolio projects	File handling
Software engineering demonstrations	PDF generation
Desktop application learning	Template systems and validation mechanisms

Conclusion

The Resume Builder project is a complete desktop application focused on structured resume creation and PDF generation. It demonstrates a solid understanding of C#, Windows Forms, layered architecture, object-oriented design, JSON-based persistence, and document-generation workflows.

Technical Area	Demonstrated Skill
Development	C# programming and Windows Forms development.
Design	Layered architecture and object-oriented programming.
Data	JSON handling and structured data management.
Output	PDF generation and user interface design.

Its modular structure makes the application easier to maintain, extend, and improve over time, while the template-based PDF engine provides a scalable foundation for future enhancement.